

Programando com textos Strings

Ana Júlia Lima



Desenvolvido para
Young 1

Ctrl Play Santa Mônica
Vila Velha, Brasil
Fevereiro 2026

Sumário

1	Strings (str)	2
1.1	O que são strings?	2
1.2	Imutabilidade (muito importante!)	2
1.3	Como criar strings	2
2	Imprimir strings (print)	2
2.1	Impressão simples	2
2.2	Imprimindo com variáveis	2
2.3	Saltando linha e acrescentando espaços	3
2.3.1	\n (quebra de linha)	3
2.3.2	\t (tabulação)	3
2.3.3	Parâmetro sep no print (separador)	3
2.3.4	Parâmetro end no print (final da impressão)	3
3	Função len()	3
4	Indexação de strings	3
4.1	Índices começam em 0	3
4.2	Índices negativos	3
4.3	Fatiamento (slicing)	4
4.4	Fatiamento com passo	4
5	Função find()	4
6	Função count()	4
7	Função str()	4
8	Métodos embutidos: upper(), lower(), split()	5
8.1	upper() (maiúsculas)	5
8.2	lower() (minúsculas)	5
8.3	split() (separar em lista)	5
9	Entrada de dados: input()	5
10	Operações com strings	5
10.1	Concatenação (juntar textos)	5
10.2	Repetição	5
10.3	Pertence? (operador in)	6
10.4	Comparação de strings	6
10.5	Formatando com f-string (recomendado)	6
11	Lista de Exercícios (Strings)	7
12	Gabarito — Lista de Exercícios (Strings)	8

1 Strings (str)

1.1 O que são strings?

String é um tipo de dado usado para representar **textos**. Em Python, strings pertencem ao tipo **str**.

```
nome = "Ana"
mensagem = 'Olá, mundo!'
frase = "Python é incrível"
```

Você pode usar:

- Aspas duplas: "texto"
- Aspas simples: 'texto'

1.2 Imutabilidade (muito importante!)

Strings em Python são **imutáveis**. Isso significa que:

- Você **não consegue alterar** um caractere dentro da string diretamente.

Exemplo do que **não** funciona:

```
nome = "Ana"
nome[0] = "a" # ERRO: strings são imutáveis
```

Como fazemos então? Criamos uma **nova** string a partir da antiga:

```
nome = "Ana"
novo_nome = "a" + nome[1:]
print(novo_nome) # "ana"
```

1.3 Como criar strings

Formas comuns de criar strings:

```
s1 = "Texto com aspas duplas"
s2 = 'Texto com aspas simples'

# String vazia
vazia = ""

# Com números (ainda é texto!)
cpf = "12345678900"
```

2 Imprimir strings (print)

2.1 Impressão simples

```
print("Olá!")
```

2.2 Imprimindo com variáveis

```
nome = "Ana"
print("Bem-vinda,", nome)
```

2.3 Saltando linha e acrescentando espaços

2.3.1 \n (quebra de linha)

\n faz o texto pular para a próxima linha.

```
print("Linha 1\nLinha 2\nLinha 3")
```

2.3.2 \t (tabulação)

\t adiciona um “tab” (um espaçamento maior).

```
print("Nome:\tAna")
print("Idade:\t18")
```

2.3.3 Parâmetro sep no print (separador)

Por padrão, o print separa itens com espaço. Você pode mudar isso com sep.

```
print("Ana", "Julia", "Lima", sep=" - ")
# Saída: Ana - Julia - Lima
```

2.3.4 Parâmetro end no print (final da impressão)

Por padrão, o print termina com quebra de linha. Você pode mudar isso com end.

```
print("Carregando...", end=" ")
print("OK!")
# Saída: Carregando... OK!
```

3 Função len()

len() retorna a **quantidade de caracteres** de uma string (inclui espaços).

```
texto = "Python"
print(len(texto)) # 6

frase = "Oi, Ana!"
print(len(frase)) # conta letras, vírgula, espaço e exclamação
```

4 Indexação de strings

Indexação é acessar um caractere usando posição (índice).

4.1 Índices começam em 0

```
palavra = "Python"
print(palavra[0]) # P
print(palavra[1]) # y
print(palavra[5]) # n
```

4.2 Índices negativos

Você pode contar do final:

```
palavra = "Python"
print(palavra[-1]) # n
print(palavra[-2]) # o
```

4.3 Fatiamiento (slicing)

Fatiar é pegar um pedaço da string:

```
s[inicio:fim]
```

Importante: o fim não entra.

```
palavra = "Python"
print(palavra[0:2]) # Py
print(palavra[2:6]) # thon
print(palavra[:3]) # Pyt
print(palavra[3:]) # hon
```

4.4 Fatiamiento com passo

```
s[inicio:fim:passo]
```

```
palavra = "Python"
print(palavra[0:6:2]) # Pto
```

Um truque famoso: inverter string:

```
palavra = "Python"
print(palavra[::-1]) # nohtyP
```

5 Função find()

find() procura um texto dentro de outro e retorna:

- o índice da primeira ocorrência, se encontrar
- -1 se não encontrar

```
texto = "Python é incrível"
print(texto.find("é")) # retorna o índice onde "é" começa
print(texto.find("incrível")) # retorna o índice do "i"
print(texto.find("Java")) # -1 (não achou)
```

6 Função count()

count() conta quantas vezes um caractere (ou palavra) aparece.

```
texto = "banana"
print(texto.count("a")) # 3
print(texto.count("na")) # 2
print(texto.count("x")) # 0
```

7 Função str()

str() converte um valor para string (texto). Isso é muito útil quando você quer juntar texto com números.

```
idade = 18
```

```
# errado: dá erro, pois não dá para somar str com int
# print("Idade: " + idade)
```

```
# certo:
print("Idade: " + str(idade))
```

8 Métodos embutidos: upper(), lower(), split()

Métodos são funções “dentro” do objeto string, por isso usamos ponto:

```
string.metodo()
```

8.1 upper() (maiúsculas)

```
nome = "Ana Julia"
print(nome.upper()) # ANA JULIA
```

8.2 lower() (minúsculas)

```
nome = "Ana Julia"
print(nome.lower()) # ana julia
```

8.3 split() (separar em lista)

split() separa um texto em partes e devolve uma **lista**.
Por padrão, separa por espaços:

```
frase = "Python é muito legal"
partes = frase.split()
print(partes)
# ['Python', 'é', 'muito', 'legal']
```

Você pode escolher o separador:

```
texto = "maçã,banana,uva"
print(texto.split(","))
# ['maçã', 'banana', 'uva']
```

9 Entrada de dados: input()

input() sempre retorna uma **string**. Mesmo que a pessoa digite número, vem como texto.

```
nome = input("Digite seu nome: ")
print("Olá,", nome)
```

Exemplo com número: precisa converter!

```
idade_txt = input("Digite sua idade: ")
idade = int(idade_txt)
print("Ano que vem você terá", idade + 1)
```

10 Operações com strings

10.1 Concatenação (juntar textos)

```
nome = "Ana"
sobrenome = "Lima"
completo = nome + " " + sobrenome
print(completo) # Ana Lima
```

10.2 Repetição

```
print("ha" * 3) # hahaha
```

10.3 Pertence? (operador in)

```
frase = "Python é incrível"
print("Python" in frase) # True
print("Java" in frase)   # False
```

10.4 Comparação de strings

Strings podem ser comparadas (ordem alfabética, baseado na tabela Unicode). Na prática, isso é mais usado para igualdade.

```
senha = "abc123"
print(senha == "abc123") # True
print(senha == "ABC123") # False
```

10.5 Formatando com f-string (recomendado)

Uma forma bem limpa de juntar texto com variáveis é usar **f-string**.

```
nome = "Ana"
idade = 18
print(f"Meu nome é {nome} e eu tenho {idade} anos.")
```

Resumo final

- Strings são textos e são **imutáveis**.
- `len()` dá o tamanho do texto.
- Indexação (`[0]`, `[-1]`) e fatiamento (`[inicio:fim]`) permitem acessar partes do texto.
- `find()` procura e retorna índice ou `-1`.
- `count()` conta ocorrências.
- `upper()`, `lower()` transformam o texto; `split()` separa em lista.
- `input()` sempre retorna string.
- Concatenação, repetição e `in` são operações muito usadas com strings.

11 Lista de Exercícios (Strings)

Nível 1 — Básico

1. Crie uma variável `nome` com seu nome e imprima:
 - O nome em maiúsculas (`upper`)
 - O nome em minúsculas (`lower`)
 - O tamanho do nome (`len`)
2. Peça ao usuário um `sobrenome` usando `input()` e imprima o nome completo usando concatenação.
3. Mostre uma frase em 3 linhas diferentes usando `\n`.
4. Faça um `print` que mostre: `Nome: <seu nome> | Idade: <sua idade>` usando `sep` para controlar o separador.

Nível 2 — Indexação e Busca

1. Dada a string `"programacao"`, imprima:
 - O primeiro caractere
 - O último caractere
 - Os 3 primeiros caracteres
 - Os 3 últimos caracteres
2. Peça ao usuário uma palavra e mostre a palavra invertida usando fatiamento (`[::-1]`).
3. Crie uma variável `texto = "banana"` e mostre:
 - Quantas vezes aparece a letra `"a"` (`count`)
 - O índice da primeira vez que aparece `"na"` (`find`)
4. Peça uma frase ao usuário e verifique se a palavra `"Python"` aparece nela usando `in`.

Nível 3 — Split e Mini-desafios

1. Peça uma frase ao usuário e:
 - Use `split()` para separar as palavras
 - Mostre quantas palavras existem (use `len()` na lista gerada)
2. Dado o texto `"maçã,banana,uva,manga"`, use `split(",")` para transformar em lista e imprima cada fruta.
3. Peça ao usuário um número (como texto) e imprima:
 - O tipo antes da conversão
 - O tipo depois de usar `int()`
 - Depois, converta de volta para string com `str()` e mostre o tipo final
4. **Desafio:** Peça um nome completo e imprima apenas o primeiro nome. Dica: use `split()` e pegue o índice `[0]`.

12 Gabarito — Lista de Exercícios (Strings)

Nível 1 — Básico

1.

```
nome = "Ana"

print(nome.upper())
print(nome.lower())
print(len(nome))
```

2.

```
nome = "Ana"
sobrenome = input("Digite seu sobrenome: ")

nome_completo = nome + " " + sobrenome
print(nome_completo)
```

3.

```
print("Linha 1\nLinha 2\nLinha 3")
```

4.

```
nome = "Ana"
idade = 18

print("Nome:", nome, "|", "Idade:", idade, sep=" ")
```

Nível 2 — Indexação e Busca

1.

```
texto = "programacao"

print(texto[0])
print(texto[-1])
print(texto[:3])
print(texto[-3:])
```

2.

```
palavra = input("Digite uma palavra: ")
print(palavra[::-1])
```

3.

```
texto = "banana"

print(texto.count("a"))
print(texto.find("na"))
```

4.

```
frase = input("Digite uma frase: ")

if "Python" in frase:
    print("A palavra Python aparece na frase.")
else:
    print("A palavra Python não aparece na frase.")
```

Nível 3 — Split e Mini-desafios

1.

```
frase = input("Digite uma frase: ")

palavras = frase.split()
print(palavras)
print("Quantidade de palavras:", len(palavras))
```

2.

```
texto = "maçã,banana,uva,manga"

frutas = texto.split(",")

for fruta in frutas:
    print(fruta)
```

3.

```
numero_txt = input("Digite um número: ")

print(type(numero_txt))

numero = int(numero_txt)
print(type(numero))

numero_str = str(numero)
print(type(numero_str))
```

4.

```
nome_completo = input("Digite seu nome completo: ")

partes = nome_completo.split()
primeiro_nome = partes[0]

print("Primeiro nome:", primeiro_nome)
```